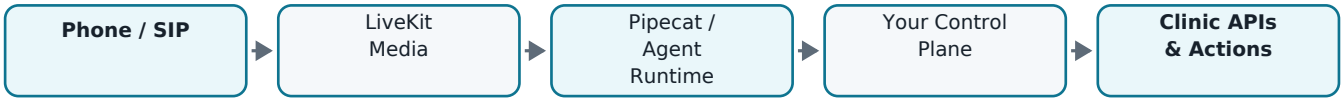
Open-Source Retell-like Voice AI Platform

Architecture Blueprint for a Multi-Tenant, Low-Latency, Scalable Voice Agent SaaS

THE PRODUCT IN ONE LINE



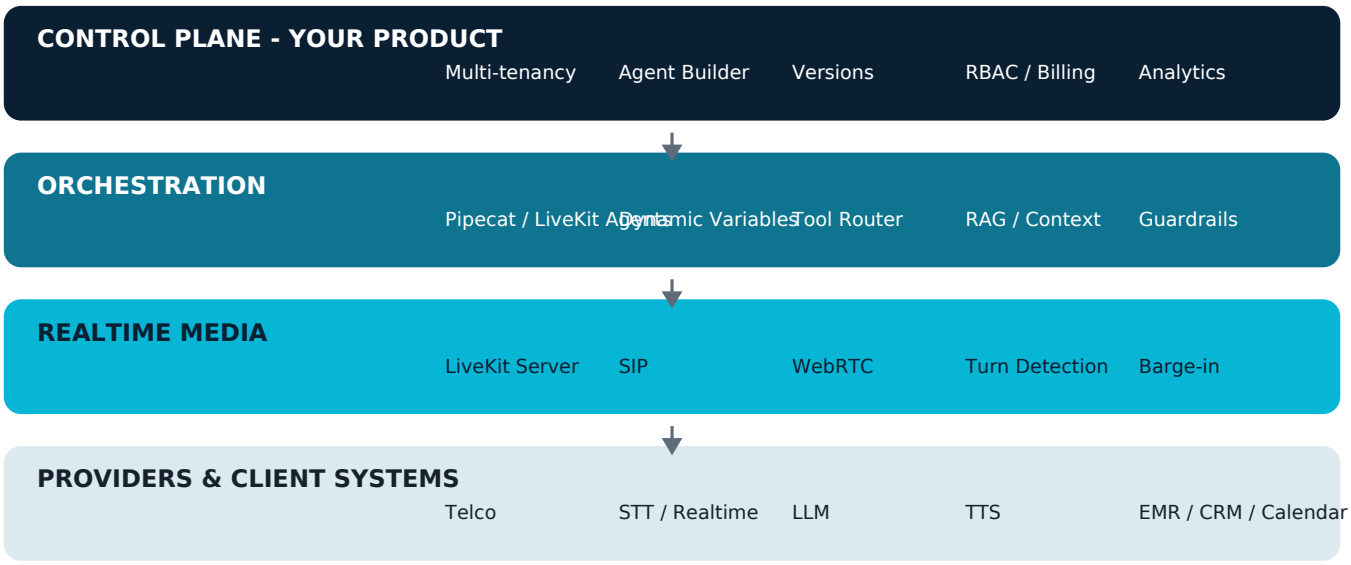
Goal: build a product that can answer calls instantly, hold natural conversations, store transcripts/recordings, use dynamic variables, call APIs, take actions, transfer calls, and later support booking/rescheduling - while remaining provider-independent and horizontally scalable.

Recommended foundation: LiveKit for realtime media/SIP + Pipecat (or LiveKit Agents after benchmark) for voice orchestration + a proprietary multi-tenant control plane for the Retell-like product experience.

Prepared for internal engineering/product discussion | Pricing snapshot: 7 Oct 2026

1. Executive Summary

We are not building a single clinic bot. We are building a reusable voice-agent platform that can serve many organizations, phone numbers, agents, workflows and integrations from one multi-tenant SaaS.

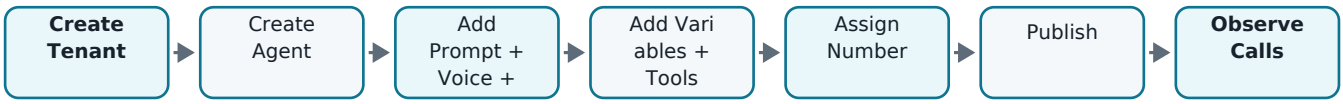


The architectural rule is simple: own the control plane and business logic; use replaceable open-source infrastructure underneath. This prevents lock-in to one model, carrier or speech provider and makes future optimization possible without rewriting the product.

Decision	Recommendation	Why
Realtime media	LiveKit	Open-source, self-hostable, SIP/WebRTC, Kubernetes/multi-region path.
Agent pipeline	Benchmark Pipecat vs LiveKit Agents	Pipecat gives strong provider abstraction; LiveKit Agents reduces operational complexity.
Telephony	SIP provider abstraction	Start Telnyx/Twilio/Exotel/Plivo depending market; do not couple agent logic to carrier.
Product layer	Build ourselves	Tenants, agents, variables, tools, KB, call logs, analytics, versions, billing, RBAC.

2. What We Are Actually Building

The platform should feel similar to Retell AI from a customer's perspective, but the underlying runtime remains modular. A client creates an organization, configures an agent, assigns a phone number, connects knowledge/tools, publishes a version and reviews calls.



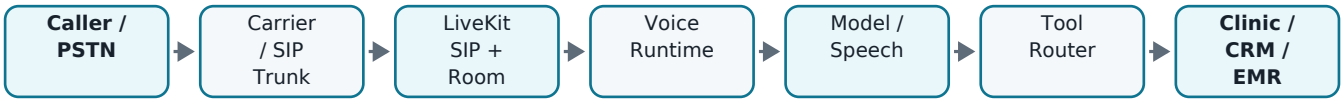
Core product objects

Object	Purpose	Key data
Organization / Workspace	Tenant boundary	users, roles, plan, limits, data policy
Agent	Voice receptionist / workflow	prompt, voice, model policy, KB, tools, fallback
Agent Version	Reproducible deployment	immutable prompt/config/tool versions
Phone Number	Inbound/outbound routing	carrier, SIP trunk, tenant, agent mapping
Call	Auditable conversation	events, transcript, recording ref, summary, cost, latency
Tool	Action/API connector	schema, auth, endpoint, timeout, response mapping

Clinic use case alignment

The supplied Always-On Clinic brief calls for an AI receptionist that answers, books, reschedules and escalates around the clock, plus fast lead response and deep scheduling/POS/EMR integration. This architecture is designed so those clinic capabilities are applications of the same generic platform rather than hard-coded one-off features.

3. End-to-End Call Architecture



A live call is a realtime event stream. Audio enters through SIP, LiveKit handles realtime transport, the runtime detects turns and interacts with the selected speech/model pipeline, and tool requests are routed through our controlled Tool Service. Every event is asynchronously written to observability/storage so logging never blocks the caller.

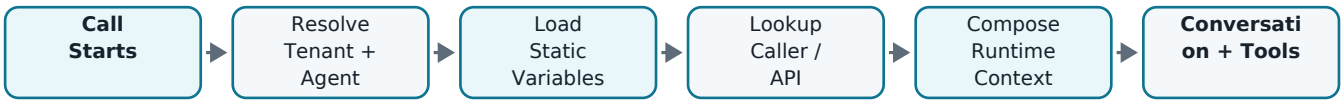
Critical separation

Realtime path - must stay fast	Async path - must never block speech
audio, turn detection, model, tool result needed for next response	transcript persistence, analytics, summaries, billing events, QA, warehouse export
small payloads, short timeouts, circuit breakers	queue/event bus, retries, batch writes, eventual consistency

Failure policy: if a non-essential service fails, the call continues. If an action API fails, the agent acknowledges the issue, avoids inventing success, retries only when safe, and can transfer/escalate.

4. Dynamic Variables + Tool/Action Engine

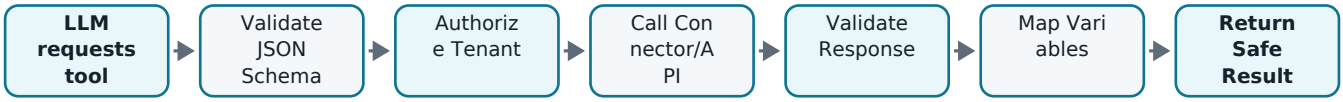
These should be first-class platform features from the beginning. They are what turns a talking bot into an operational agent.



Variable classes

Type	Examples	Resolution
Static	clinic_name, address, services	agent/tenant configuration
Runtime	call_id, caller_number, current_time	session metadata
API-derived	patient_name, next_appointment	tool/API lookup with cache/TTL
Computed	days_since_visit, eligible_for_rebooking	deterministic rule engine

Generic Tool Service



Tool definitions should include name, description, JSON input/output schemas, endpoint/connector, authentication reference, timeout, retry policy, idempotency key, response mapping and audit policy. Appointment booking, rescheduling, CRM lead creation and SMS then become configurations/connectors - not custom voice-engine code.

5. Multi-Tenant Data & Service Architecture

Every request, call, agent, tool execution and stored artifact must be scoped by tenant_id. Tenant isolation is an architectural primitive, not a later feature.

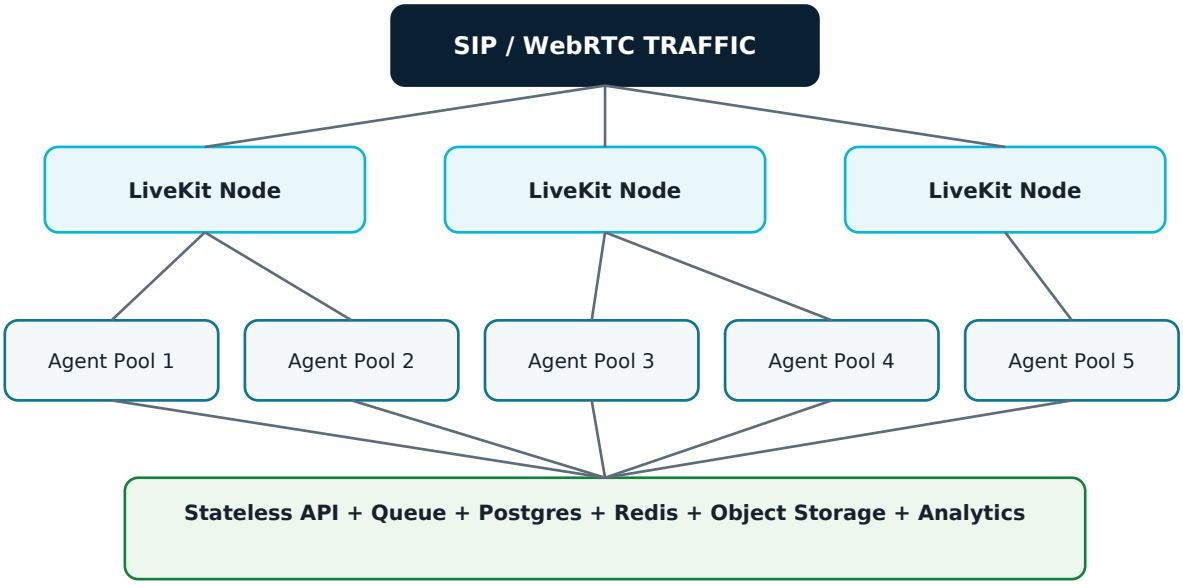


Service / Store	Recommended responsibility
PostgreSQL	organizations, users, agents, versions, tools, integrations, calls, billing metadata
Redis	session state, short-lived variables, rate limits, locks, caches
Object storage	recordings and large artifacts where consent/policy permits
Queue / event bus	transcript writes, summaries, evaluations, webhooks, usage aggregation
Analytics store	high-volume call/turn/tool/latency metrics; add ClickHouse or equivalent when needed
Secrets manager	tenant connector credentials and provider keys; never store raw secrets in agent configs

Recommended database principle: start with Postgres + Redis + object storage. Add specialized infrastructure only after measurements justify it. Premature complexity is not scalability.

6. How It Scales to 10,000+ Users

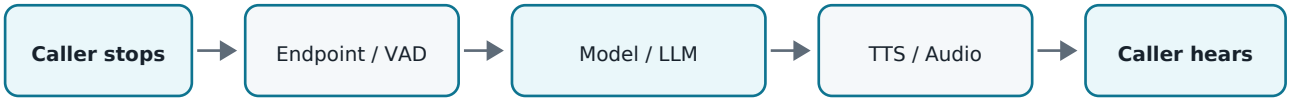
Registered users are not the capacity metric. Concurrent calls and turns per second are. The architecture therefore scales horizontally around realtime sessions.



Stage	Illustrative concurrency target	Architecture move
Launch	10-50 calls	small cluster, managed DB, one region, autoscaled agent workers
Growth	50-250 calls	separate worker pools, queue consumers, DB read tuning, autoscaling
Scale	250-1,000 calls	multi-node LiveKit, regional workers, analytics store, provider quotas
Large scale	1,000-5,000+ calls	multi-region routing, capacity reservations, carrier/model redundancy, sharded hot paths as required

LiveKit's self-hosted model supports Kubernetes and distributed multi-region deployments; its agent servers use worker pools and job dispatch across available workers. That is why it is a strong media foundation even if we use Pipecat for orchestration.

7. Latency Strategy: How We Compete on Conversation Quality



Instrument every turn:

endpoint_ms | stt_final_ms | llm_ttft_ms | tts_ttfb_ms | first_audio_ms | tool_ms

Primary KPI: P50/P95 time-to-first-useful-audio. Optimize the slowest measured stage, not intuition.

We should not promise a universal millisecond number before testing real PSTN routes, languages, models and tools. Instead, establish a latency SLO from production measurements and optimize P50/P95 time-to-first-useful-audio.

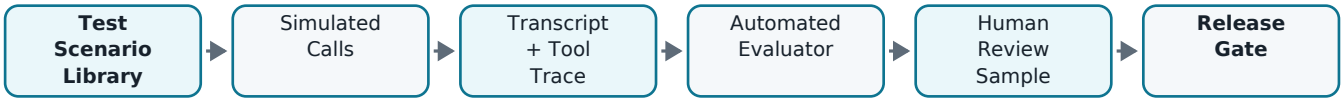
Optimization	Implementation
Place compute near callers/providers	regional LiveKit/agent workers; avoid unnecessary cross-region hops
Streaming everywhere	stream audio/STT/model/TTS; never wait for whole utterance or response where avoidable
Fast endpointing	tune VAD/turn detection by language and call type; support barge-in
Short critical path	keep transcript/analytics writes async; tool calls use strict timeouts
Provider routing	benchmark providers per geography/language and keep fallback choices
Warm capacity	avoid cold starts for realtime workers; autoscale before saturation

Decision gate: run the same 100-200 test calls through (A) LiveKit Agents and (B) Pipecat + LiveKit transport using the same carrier/model. Select on measured latency, interruption behavior, CPU/RAM, failures and engineering complexity.

8. Accuracy & Reliability Strategy

Accuracy is not only model intelligence. It comes from controlling which source is authoritative for each type of answer.

Question type	Source of truth	Rule
Hours, services, parking, policies	Knowledge base / structured tenant data	retrieve, cite internally, do not invent
Caller/patient context	dynamic variables / lookup API	refresh when stale
Availability, booking, cancellation	authoritative scheduling API	tool call required before confirmation
High-risk / unsupported request	guardrail + escalation policy	do not improvise; transfer/escalate



Create regression tests for every agent version. Examples: business-hours question, noisy caller, interruption, ambiguous appointment request, API timeout, duplicate booking attempt, transfer failure and safety/escalation scenario. A prompt/model/tool change should not reach production until it passes the release suite.

For healthcare deployments, recordings/transcripts may contain protected health information. Consent, retention, encryption, access controls, audit logs, BAAs/provider agreements and jurisdiction-specific recording rules must be reviewed before production.

9. Tools & Technology We Will Need

Layer	Primary choice	Alternative / note
Realtime media	LiveKit	self-host or Cloud; SIP + WebRTC
Voice orchestration	Pipecat benchmark	compare with LiveKit Agents before locking
Telephony	Telnyx / Twilio / Exotel / Plivo	carrier abstraction by geography
Speech/model	provider abstraction	native realtime or STT + LLM + TTS
Backend	Python FastAPI or Node/NestJS	pick team strength; runtime services stay stateless
Frontend	Next.js / React	agent builder, calls, analytics, settings
Database	PostgreSQL	tenant-scoped relational control plane
Cache/session	Redis	session state, locks, rate limits
Storage	S3-compatible object storage	recordings/artifacts under retention policy
Async jobs	Redis Streams / NATS / Kafka later	start simple; evolve with throughput
Observability	OpenTelemetry + Prometheus/Grafana	traces, latency, errors, provider metrics
Deployment	Docker; Kubernetes when justified	separate dev/staging/prod
Secrets	cloud secrets manager / Vault	tenant/provider credentials

10. Estimated Cost

These are planning ranges, not vendor quotes. Actual spend depends primarily on concurrent calls, call minutes, geography, chosen model, recording retention and whether infrastructure is self-hosted or managed.

A. Build cost - 2-week V1

Team	Assumption	Indicative cost
2 backend/voice engineers	2 weeks each	$4,000-$9,600 at $25-$60/hr
1 frontend/full-stack engineer	2 weeks	$2,000-$4,800
DevOps/QA/Product support	~1 person-week combined	$1,000-$2,400
Engineering total	~7 person-weeks	~$7,000-$16,800 external-equivalent

If the team is already salaried internally, treat this as opportunity-cost rather than cash spend. A production-polished Retell-like platform is a multi-month product; the 2-week target is a sellable/usable V1 foundation, not feature parity.

B. Early monthly infrastructure

Item	Early-stage planning range	Notes
App + realtime compute	$150-$600/mo	small redundant deployment; increases with concurrency
Managed Postgres/Redis	$50-$250/mo	can start lower; prioritize backups
Storage/observability	$25-$200/mo	recording volume/retention drives storage
Development/test calls	$50-$500/mo	depends on test volume and providers
Baseline	~$275-$1,550/mo	before substantial production call usage

11. Per-Minute Economics & Cost Control

Open source removes platform markup, not telephony or inference cost. Current official pricing illustrates the order of magnitude: OpenAI lists GPT-Live 1 voice sessions at $0.05/min; Telnyx lists Voice API at $0.002/min plus SIP trunking, with US example trunk rates from roughly $0.0032 inbound / $0.005 outbound; provider/model rates vary by destination and usage.

Architecture	Planning range	Use when
Native speech-to-speech	~$0.06-$0.10+/call-min	fastest path to excellent natural interaction; verify actual model/carrier usage
STT + LLM + TTS	~$0.04-$0.12+/call-min	more provider control; economics depend heavily on TTS/LLM/context
Self-hosted models	GPU-capacity economics	only after sustained volume makes utilization predictable

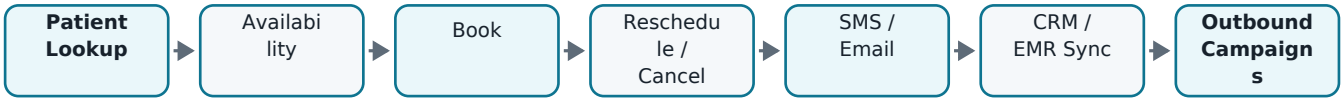
Example planning model: at 100,000 call-minutes/month, every $0.01/min changes monthly variable cost by $1,000. Therefore provider routing, prompt/context efficiency, silence handling, caching and negotiated volume pricing become material once usage grows.

Do not self-host speech/LLM models solely to make the stack 'free' at launch. Idle GPUs can cost more than APIs. First reach stable utilization and collect real cost-per-successful-call data.

12. 14-Day V1 Delivery Plan

Days	Deliverable	Exit criterion
1-2	LiveKit/SIP + two runtime POCs	real PSTN call works; benchmark instrumentation active
3-4	tenant + agent config + publish/version model	two tenants can run isolated agents
5-6	call events, transcript, recording refs, summary	call is fully reconstructable
7-8	knowledge + dynamic variables	tenant-specific answers/context
9-10	generic Tool Service + transfer	safe API tool executes and audit trail is stored
11-12	admin dashboard + latency/cost view	team can inspect calls and diagnose failures
13-14	load/failure/accuracy tests + pilot	release checklist passed; pilot number live

V2 - actions and platform depth



V2 should add connectors and workflows without changing the media/runtime architecture. This is why the generic Tool Service and dynamic-variable engine are part of the V1 foundation.

13. Key Risks and Design Decisions

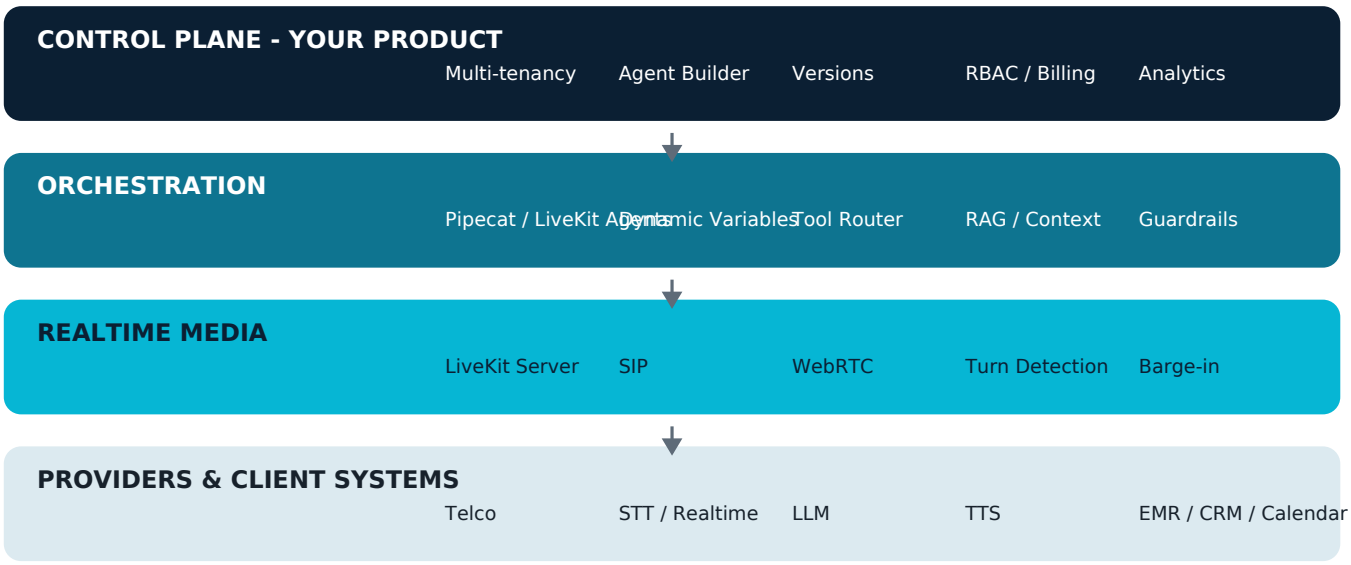
Risk	Mitigation
Provider lock-in	adapter interfaces for carrier, realtime model, STT, TTS and LLM; keep business logic outside provider SDKs
Latency regressions	per-turn traces, P50/P95 SLOs, canary agent versions, regional routing
Hallucinated actions	authoritative tool calls, schema validation, idempotency, explicit success/failure states
Noisy production calls	noise handling, interruption tests, fallback/transfer path, real PSTN testing
Scaling bottleneck	stateless workers, queues, autoscaling, concurrency dashboards, provider quota monitoring
Healthcare privacy/compliance	consent, encryption, retention controls, audit logs, access policies, BAA/vendor review; legal review by jurisdiction
Cost surprise	cost per call/tenant/provider, budgets, quotas, usage alerts, model routing

Recommended product principle

LiveKit and Pipecat are infrastructure, not the product. The durable asset is our control plane: multi-tenancy, agent lifecycle, variables, tool/action engine, integrations, observability, evaluation, billing and developer API.

14. Architecture Decision for the Team

Recommended target architecture



Now	Later
LiveKit media/SIP; benchmark Pipecat vs LiveKit Agents; Postgres/Redis; generic Tool Service; basic dashboard; one clinic integration ecosystem.	multi-region, advanced analytics/evals, connector marketplace, outbound campaigns, model routing, enterprise controls, deeper EMR/CRM integrations.

Go/no-go benchmark before final runtime choice

Use identical carrier, model, prompt and test scenarios. Compare P50/P95 first-audio latency, barge-in quality, false turn endings, call failures, CPU/RAM per concurrent call, tool latency, transcript quality and implementation complexity. The winner becomes the default runtime, but provider interfaces remain replaceable.

Bottom line: this architecture can launch cheaply, avoid rebuilding the core at growth stage, and scale by adding realtime nodes/worker capacity rather than redesigning the product. The team should optimize for concurrent-call economics and measured conversational quality, not raw user count.

15. Sources & Pricing Notes

Official/public sources used for architecture and current pricing checks. Prices change; re-check before procurement or client pricing.

LiveKit Agents
https://docs.livekit.io/agents/
Realtime agent framework; Python/Node; provider-agnostic pipelines.

LiveKit self-hosting
https://docs.livekit.io/transport/self-hosting/
Kubernetes, multi-region, self-hosting and telephony deployment model.

LiveKit SIP
https://docs.livekit.io/telephony/
SIP telephony and tested provider integrations.

LiveKit agent deployments
https://docs.livekit.io/deploy/custom/deployments/
Worker pools and job dispatch across agent servers.

Pipecat LiveKit transport
https://reference-server.pipecat.ai/en/latest/api/pipecat.transports.livekit.transport.html
Pipecat integration with LiveKit transport.

OpenAI API pricing
https://developers.openai.com/api/docs/pricing
Current GPT-Live and realtime/audio pricing.

OpenAI voice cost guide
https://developers.openai.com/api/docs/guides/voice-latency-cost
How voice session/realtime usage is measured.

Telnyx Voice API pricing
https://telnyx.com/pricing/voice-api
Illustrative telephony/SIP/recording/STT rates.

Cartesia pricing
https://www.cartesia.ai/pricing
Illustrative TTS/STT/managed-agent pricing and concurrency tiers.

Source document alignment: The Always-On Clinic (CBB Coaching & Consulting / Avishkar Solutions, 5 Oct 2026) - particularly the AI receptionist, speed-to-lead, integration and compliance requirements supplied for this discussion.